Project - Game of Life

Robin De Vos (MEIECFS)
Anthony Tacquet (MEISWE)

May 2026



1 Context

We decided to make Conway's Game of Life as the goals of this project. Everything in this game occurs in a single world, which is a two dimensional area. Each pixel in this world represents "life" or "death". This world knows time and according to some rules (which we will explain later) a tile/entity can stay alive, die, resurrect or stay dead.

In general what happens is, for each tile we calculate the amount of neighbours this tile has (between 0 and 8), and then we apply some rules based on how many neighbours this tile has. We keep doing this until we quit the game, each such iteration we call a "life cycle".

1.1 The Rules of The Game

We have four rules that decide the "fate" of a tile:

1. If a tile has **less** than 2 neighbours, the tile we are observing will die in the next life cycle due to under population.
2. If a tile has **more** than 3 neighbours, the tile will also die in the next life cycle but now due to overpopulation.
3. If a tile has exactly 3 neighbours our tile—if currently alive—will stay alive or—if currently dead—resurrect.
4. The last possibility, a tile has exactly 2 neighbours it will maintain its current state in the next life cycle (so if currently dead, stay dead, if currently alive, stay alive).

2 Methodology

Now we will discuss the structure and layout of our program, this section focusses on the GPU. For the CPU we have similar functions that we used for the ones discussed in section 2.1. On our CPU we both tried using one-dimensional (flatten) and two-dimensional arrays as representation of our world. For our measurements in section 4 we used a two-dimensional array for the CPU since the performance was similar.

2.1 GPU kernels

Init rng
This is a basic kernel function we use to initialize our states to make the random number generation work, this kernel uses the Curand library. We were forced to use this kernel, since this was the only way we found for generating random values on the GPU.

Use rules
This kernel function applies the rules we discussed earlier. For this we use a chain of

if-else statements. See listing 2 in the appendix.

Calculate neighbours
In this kernel function we first loop through all the neighbouring pixels and count the neighbours whilst also taking the sum of their colour values. We provided a code snippet in the appendix, refer to listing 3.

2.2 GUI features

Beside our GPU kernels the program also has some CPU function that have to do with the initial conditions of the world.

Generate random
One of these features is the generate random function which generates a random noise pattern of both alive pixels and dead ones. it also makes the alive pixels have a random colour. This function can be called at run time with the R-key.

Drawing
We also implemented the option to draw onto our world to make this 0 player game somewhat intractable. The drawing currently doesn't have randomised colours it uses a fixed shade of green. To have your drawing not disappear instantly we also implemented a pause button which is bound to the Space-key.

Clear
Another important feature is the option to clear the screen which can also be done using the C-key.

3 Bonus features

We decided to implement some extra features to make the project a bit more interesting, so we added a colour and an image feature.

3.1 Colour feature

This feature gives each living entity a colour value, this makes our game a lot more interesting to watch.

How we determine the colour of entities in following lifecycles?

In our neighbour calculation we originally took the average of the red, green and blue values of each of our living neighbours and use these average RGB values as colour for our entity in the next lifecycle.

When we tested this feature as is, our colours became 'dull', this was due to averaging. We tackled this problem by adding colour deviation, for each RGB value we added a

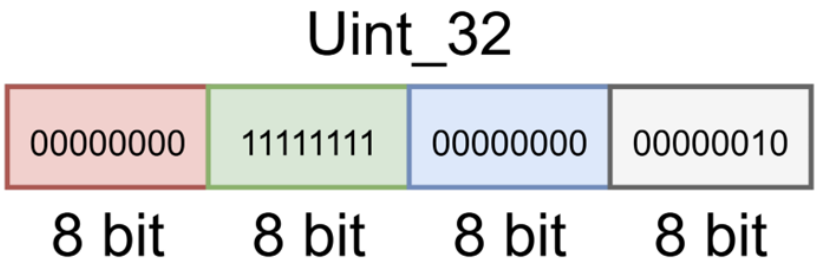


Figure 1

random value between -20 and +20.

How do we store colour information for our entities?

We wanted to keep our code as fast as possible so we used a clever way (if we say so ourselves) to store the colour values for each entity in an efficient way. We use 32 unsigned integers to store data for each pixel. We store the data such that the first 8 bits contain the value for red, the next 8 pixels green and then blue. In the last 8 bits we store the amount of neighbours. This is show in figure 1. In order to access these values we use shifting operations.

3.2 Image feature

This feature seemed interesting at first, but in hindsight the result was not what we expected it to be. We simply used the .ppm image of a previous lab assignment and used its array of colour values in our project.

What we observed when loading the image is obvious when you think about it, but we did not really think about it very thoroughly. All entities of the image die due to over-population in the lifecycle after loading in the image. Luckily new entities are resurrected at the edges of where the image used to be because they all had 3 neighbours when the neighbours were calculated.

In the following lifecycle we have two lines of entities (left and right of where the image used to be), on both sides of each line a series of new entities is resurrected (3 neighbours) this results in a thicker line of 3 entities wide. In the following lifecycle the two original lines die, again due to overpopulation and we end up with 4 lines now (because of resurrection).

This process keeps repeating and we get this ‘glitch-like’ effect shown in figure 2.

4 Observations/results

As is clear from table 1 the GPU is clearly dominant in the calculation of the neighbours and the fate of each entity. Even for the smaller matrix sizes we would have expected the

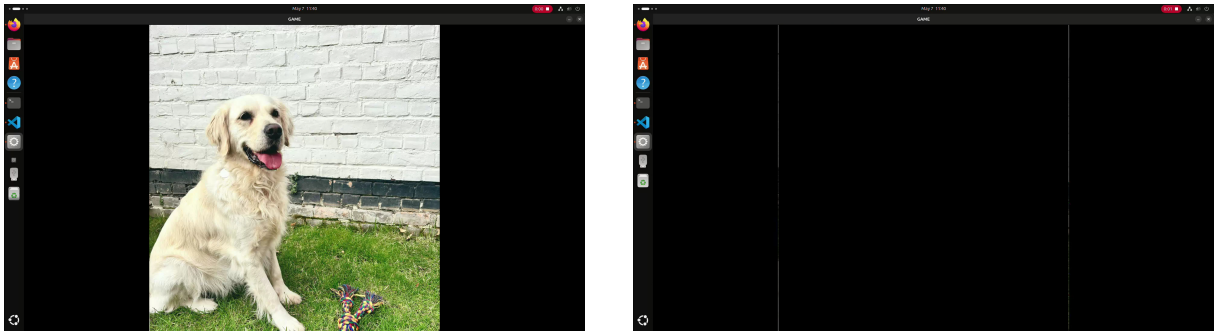


Figure 2: First lifecycle (on image load) and second lifecycle (all entities dead, newly resurrected stripes)

CPU to be dominant but nonetheless our GPU code performed better. This might be due to our poor programming or just because it deemed to be so.

Table 1: GPU vs CPU performance comparison

Model/Dimension	**10**	**100**	**250**	**500**	**1000**	**2500**	**5000**
GPU	0.007979	0.220551	0.120883	0.256701	0.822701	5.03831	20.2473
CPU	0.033511	2.11182	12.305	48.8585	194.178	-	-

The "-" means that we did not perform this measurement, because it was taking up a lot of time.
The top row represents the dimension of the matrix, for example 10 means that our world has a dimension of 10x10.
The measurements are expressed in seconds

What is shown in table 1 can also be plotted in a graph shown in figure 3. Here you can clearly see how dominant the GPU is for this task.

5 Conclusion

In conclusion, the CPU implementation performs adequately for smaller data sizes (up to 100x100), but is significantly outperformed by the GPU implementation as the scale increases. This performance gap is expected. Furthermore, while the CPU approach benefits from a simpler design, the GPU implementation manages to deliver superior performance without introducing excessive complexity.

5.1 Improvements

Our current rendering is very inefficient. We currently draw our cell by having a rectangle being drawn for each living cell. A more efficient way of tackling this issue would be to create textures of the living cells and render this once each frame in stead of drawing 25 million rectangles (if we use a 5000x5000 matrix).

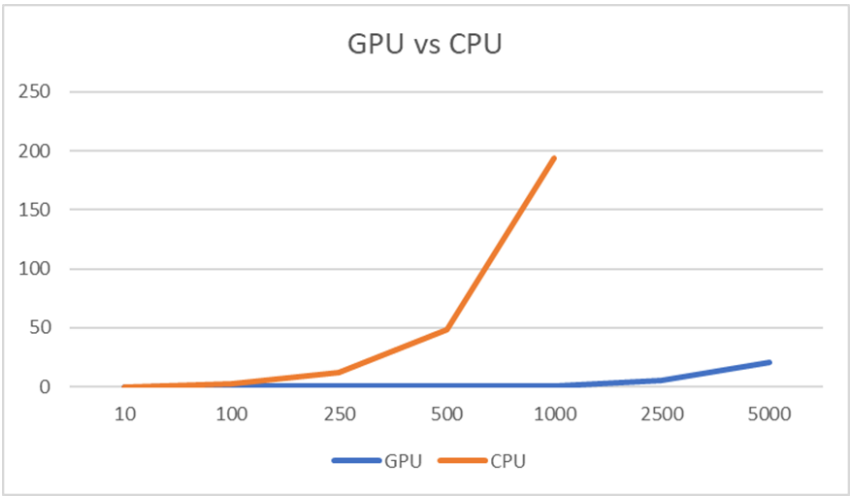


Figure 3
the y-axis shows us the processing time in seconds for a 1000 lifecycles
the x-axis shows us the dimension of the world, N for N x N matrix

Another useful feature would have to be a zoom function since we need large world sizes to have sufficient load on our GPU. At these large world sizes it can be hard to verify if the rules are being applied correctly.

Another optimisation we could implement has to do with colour calculation. The current kernel always calculates the colour of each pixel even if that pixel is dead and will remain to be dead in the next lifecycle, this probably slows down our performance.

We also found that the modulo operation is slow on GPU and a possible solution for this could be to use a technique called branch wrapping.

Last improvements are minor improvements that have to do with the image function. First improvement would be to make the image size flexible as this is currently hardcoded. A second improvement would be to use a more common format beside the .ppm which it currently uses. Lastly, implement a way to make it so that darker coloured pixels don't get imported (from an image) as living cells with a black colour but instead as dead cells. These improvements would create a more versatile image loading function.

A Code Listings

Listing 1: Global Variables

```
#define M 960        // VR width
#define N 1280        // VR height
#define C 3          // Colors
#define OFFSET 16    // Header length

#define length 1080
```

```
#define width 1920

#define window_x 1920
#define window_y 1080

#define MASK_R 0xff000000
#define MASK_G 0x00ff0000
#define MASK_B 0x0000ff00
#define MASK_N 0x000000ff
```

Listing 2: Use Rules Kernel

```
__global__ void useRules(u_int32_t *current_doa, u_int32_t *
    neighbours, u_int32_t *next_doa) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx >= length * width) return;

    u_int32_t pixel_neighbour = neighbours[idx] & MASK_N;

    if (pixel_neighbour < 2 || pixel_neighbour > 3) {
        next_doa[idx] = 0;
    } else if (pixel_neighbour == 3) {
        // Always stay alive if neighours == 3
        next_doa[idx] = neighbours[idx];
    } else {
        // If pixel_neighbour == 2
        next_doa[idx] = current_doa[idx];
    }
}
```

Listing 3: Calculate Neighbours Kernel

```
__global__ void calculateNeighbours(u_int32_t *current_doa,
    u_int32_t *neighbours, int *offsets, curandState *states) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx >= length * width) return;

    int x = idx % width;
    int y = idx / width;

    u_int32_t avg_R = 0;
    u_int32_t avg_G = 0;
    u_int32_t avg_B = 0;

```

```
12      for(int i = 0; i < 16; i += 2) {
13          int ox = offsets[i];
14          int oy = offsets[i + 1];
15
16          int nx = (x + ox + width) % width;
17          int ny = (y + oy + length) % length;
18
19          if (current_doa[nx + ny * width] > 0) {
20              neighbours[idx] += 1;
21              avg_R += (current_doa[nx + ny * width] & MASK_R) >>
                    24;
22              avg_G += (current_doa[nx + ny * width] & MASK_G) >>
                    16;
23              avg_B += (current_doa[nx + ny * width] & MASK_B) >>
                    8;
24          }
25      }
26
27      u_int32_t count = neighbours[idx]; // save before overwrite
28      if (count > 0) {
29          curandState localState = states[idx];
30
31          int var_R = (curand(&localState) % 41 - 20);
32          int var_G = (curand(&localState) % 41 - 20);
33          int var_B = (curand(&localState) % 41 - 20);
34
35          int val_R = (avg_R / count) + var_R;
36          int val_G = (avg_G / count) + var_G;
37          int val_B = (avg_B / count) + var_B;
38
39          states[idx] = localState; // save back state
40
41          neighbours[idx] = ((val_R < 255 && val_R > 50 ? val_R :
                val_R < 50 ? 60 : 235) << 24)
42          | ((val_G < 255 && val_G > 50 ? val_G : val_G < 50 ? 60 :
                235) << 16)
43          | ((val_B < 255 && val_B > 50 ? val_B : val_B < 50 ? 60 :
                235) << 8)
44          | count; // preserve count in low byte for useRules
45      } else {
46          neighbours[idx] = 0;
47      }
48  }
```